

Programação para Dispositivos Móveis

Conteúdo:

- Apresentação da disciplina: Plano de Ensino
- Introdução.

Material elaborado pelos professores: **Ledón / Alcides**



Coordenador do Curso de **CCP**

Prof. Claudio Benossi

cbenossi@cruzeirodosul.edu.br

Contato

Coordenação de Informática
Campus São Miguel Paulista
Coordenação - F. 2037-5751



Coordenador dos Cursos de **ADS** e **GTI**

Prof. Giulio Guiyti Rossignolo Suzumura

giulio.suzumura@cruzeirosul.edu.br

Contato

Coordenação de Informática
Campus São Miguel Paulista
Coordenação - F. 2037-5751



Coordenação nos campi

Coordenação em cada campus

Campus São Miguel Paulista - Prof. Giulio - giulio.suzumura@cruzeirodosul.edu.br

Campus Anália Franco - Prof. Marco Antonio - marco.sanches@cruzeirodosul.edu.br

Campus Guarulhos - Prof. Claudio Benossi - cbenossi@cruzeirodosul.edu.br



Informações públicas em:
<https://buscatextual.cnpq.br>

Professor da disciplina

Prof. MSc. Manuel Fernández Paradela **Ledón**



- Graduação em Engenharia Elétrica (Computação)
- Mestrado em Informática Aplicada na Engenharia e Arquitetura

Universidad Tecnológica de La Habana 'José Antonio Echeverría'.
Diplomas revalidados na Universidade de São Paulo, USP.

Contato

manuel.ledon@cruzeirosul.edu.br
mfpledon@gmail.com

<https://sites.google.com/site/mfpledon/info-geral>
<http://mfpledon.com.br>



Horário das aulas

Manhã			
1h15m por módulo	1º Módulo	Intervalo (10 min)	2º Módulo
	8h30 às 9h45	9h45 às 9h55	9h55 às 11h10
Noturno			
1h15m por módulo	1º Módulo	Intervalo (10 min)	2º Módulo
	19h10 às 20h25	20h25 às 20h35	20h35 às 21h50

Total da aula: 150 minutos

http://mfpledon.com.br/horarios_das_aulas.png

Alguns sites da Universidade Cruzeiro do Sul

Portal - Universidade Cruzeiro do Sul

www.cruzeirosul.edu.br

Blackboard

bb.cruzeirosulvirtual.com.br

Pós-graduação

www.cruzeirosul.edu.br/pos-graduacao

Área do Aluno, CAA, Biblioteca etc.

<https://novoportal.cruzeirosul.edu.br/?empresa=cruzeiro>



Google Play

Jogos

Apps

Filmes



Duda

Cruzeiro do Sul Educacional SA

4,8★

10,4 mil avaliações

100 mil+

Downloads

L

Classificação Livre ⓘ

Instalar



Este app está disponível para todos os seus dispositivos

Plano de ensino - Ementa

Estudo de técnicas, linguagens de programação e ferramentas para desenvolvimento de aplicações para dispositivos móveis.

Conteúdos

Apresentação da Disciplina (Plano de Ensino)

Android – Parte I

Introdução ao desenvolvimento para Android

Ferramentas para desenvolvimento

Android SDK

Layouts, pacotes e classes básicas para desenvolvimento

Android – Parte II

Intent e suas aplicações.

Atenção a eventos.

Menus em Android.

Solicitações HTTP.

Processamento de bancos de dados em programas Android.

Conteúdos

Android – Parte III

Aplicações com interfaces gráficas.

Processamento de XML/JSON com Android.

Outras linguagens para desenvolvimento

Introdução ao desenvolvimento mobile com a linguagem de programação Kotlin.

Desenvolvimento multiplataforma

Ferramentas atuais do mercado.

Exemplos de aplicações.

Conversão entre plataformas.

Flutter.

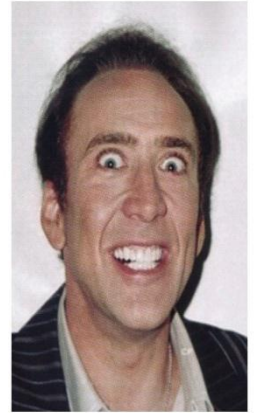
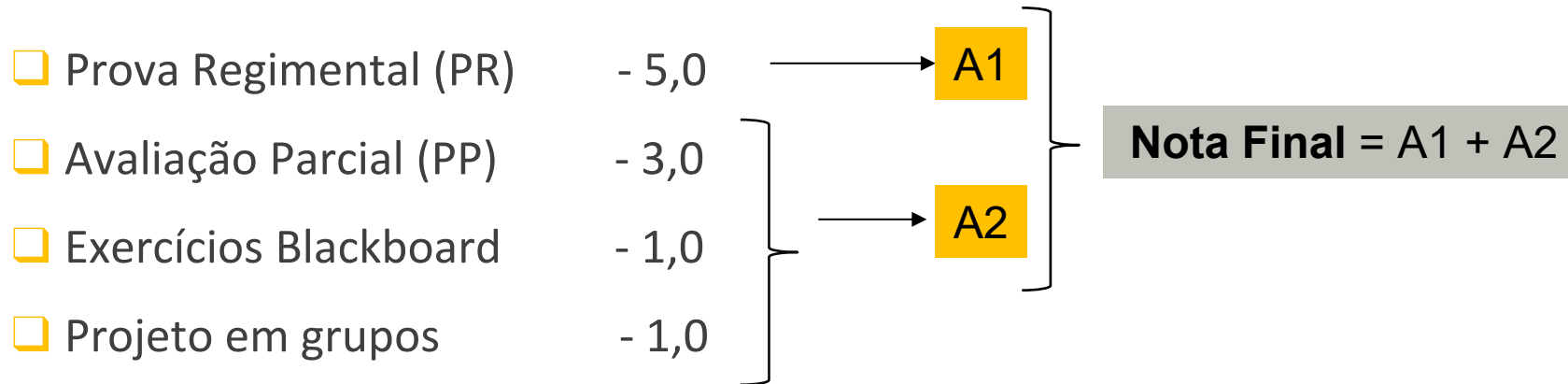
Conteúdos, segundo o Plano de Ensino

UNID.	C/H	CONTEÚDO
I	3	Apresentação Apresentação e discussão do Plano de Ensino, focando objetivos, conteúdos, estratégias, avaliação e bibliografia. Primeiros conteúdos: plataformas para desenvolvimento, tecnologias e ferramentas, linguagens.
II	12	Conceitos iniciais Introdução ao desenvolvimento para Android. Ferramentas para desenvolvimento. Android SDK. Interfaces. Layouts, pacotes e classes para desenvolvimento. Internacionalização.
III	18	Desenvolvimento em Android Intent e suas aplicações. Atenção a eventos. Menus em Android. Processamento de listas. Solicitações HTTP. Processamento de bancos de dados em programas Android.
IV	15	Interfaces gráficas. Processamento JSON ou XML Aplicações com interfaces gráficas. Processamento de JSON e/ou XML com Android.
V	3	Desenvolvimento multi-plataforma Desenvolvimento multi-plataforma. Ferramentas atuais do mercado. Exemplos de aplicações. Utilização de plataformas ou frameworks. WebView. Flutter. Dart.
VI	3	Linguagens Java e Kotlin Introdução ao desenvolvimento para Android com as linguagens de programação Java e Kotlin.
VII	6	Avaliações Avaliações: Provas, exercícios e projetos da disciplina.

Objetivos, segundo o Plano de Ensino

OBJETIVOS	
<i>Cognitivos</i>	- Conhecer os paradigmas de desenvolvimento de aplicações para dispositivos móveis; - Conhecer processos de elaboração de aplicações mobile; - Estudar diferentes tecnologias, linguagens e ferramentas para programação e desenvolvimento de aplicações em dispositivos móveis.
<i>Habilidades</i>	- Utilizar os conceitos teóricos estudados em domínios de aplicação para tecnologias específicas de desenvolvimento; - Identificar os tipos de ferramentas de desenvolvimento para aplicações móveis e aplicar a problemas reais; - Utilizar os novos conceitos e ser capaz de acomodá-los de acordo com o próprio conhecimento; - Desenvolver diferentes soluções para dispositivos móveis.
<i>Atitudes</i>	- Ter desenvoltura e segurança na utilização das linguagens de programação e tecnologias estudadas; - Ser crítico, receptivo e estar preparado para o trabalho em equipes ou coletivos de pesquisadores e programadores; - Ser analítico e responsável; - Solidificar o pensamento abstrato; - Desenvolver o interesse pela pesquisa e pelo conhecimento de novas tecnologias; - Ser criativo e ter iniciativa diante da solução de problemas.

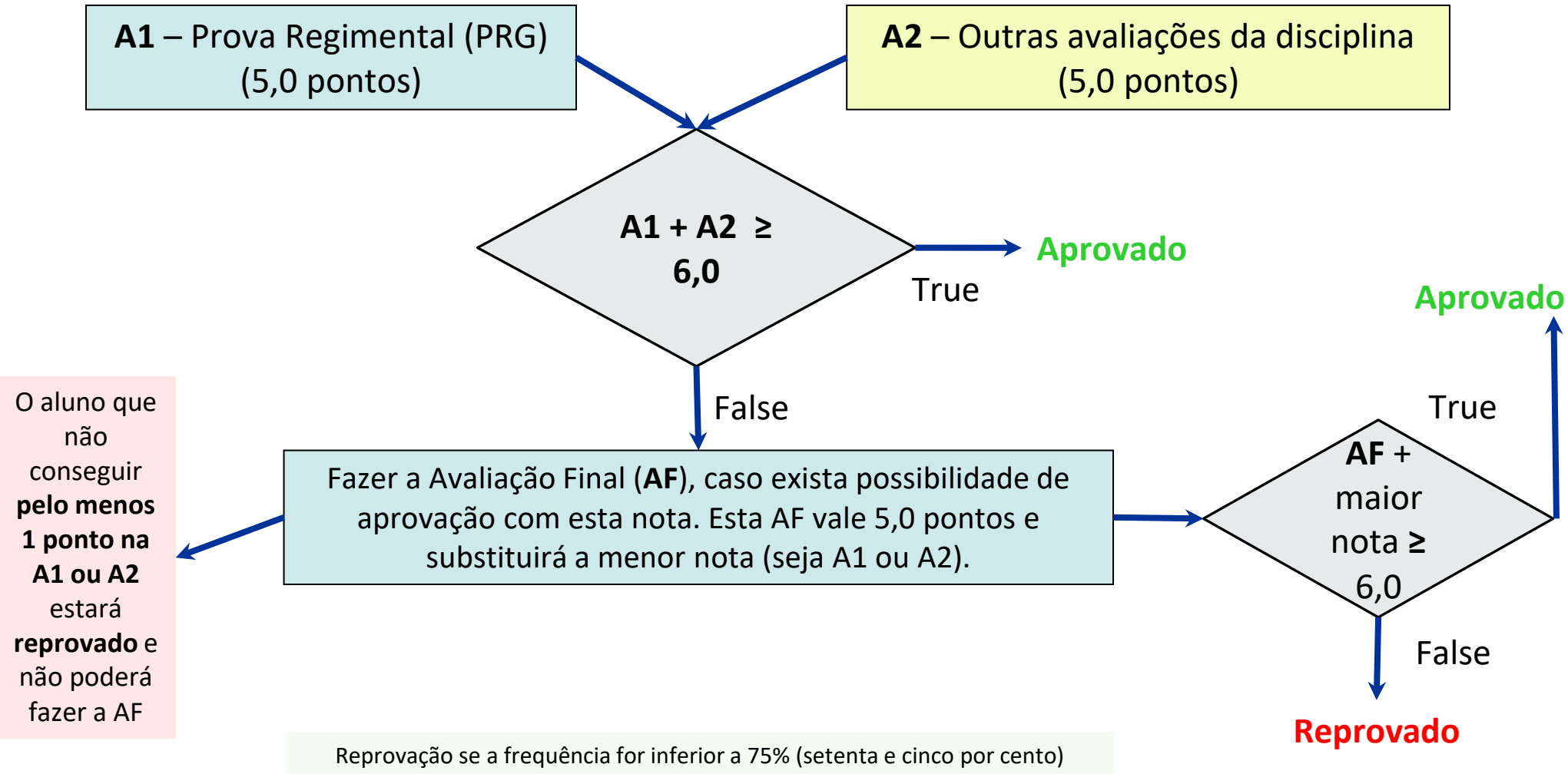
Avaliação da disciplina



- ❑ Todas as avaliações desta disciplina serão avaliações teórico/práticas, utilizando computadores e softwares.
- ❑ Os horários de início e término das avaliações serão determinados pelos professores.
- ❑ Obs: o sistema de notas só arredonda a **Nota Final**.



Sistema de avaliação



Bibliografia oficial da disciplina

BIBLIOGRAFIA BÁSICA	BIBLIOGRAFIA COMPLEMENTAR
DEITEL, H.; DEITEL, P.; DEITEL, A. Android: como programar. 2a. ed. Porto Alegre: Bookman, 2015. ISBN 978-85-8260-348-2. E-book.	W3SCHOOLS. W3schools.blog. Android Tutorial. Disponível em: https://www.w3schools.blog/android-tutorial . Acesso em: 24 mai. 2023.
OLIVEIRA, Diego B. de, et al. Desenvolvimento para dispositivos móveis. v 1 e 2. Porto Alegre: SAGAH, 2019. ISBN 978-85-9502-940-8. E-book.	SIMAS, V. L. Desenvolvimento para dispositivos móveis : volume 2. Porto Alegre: SAGAH, 2019. E-book.
MORAIS, Myllena S. de F. et. al. Fundamentos de Desenvolvimento Mobile. Porto Alegre: SAGAH Educação S.A., 2022. E-book.	FÉLIX, R. Arquitetura para computação móvel. 2. ed. São Paulo: Pearson, 2019. E-book.
	DEVELOPER. Desenvolvedores Android Android Developers. GOOGLE Android Developers. Disponível em https://developer.android.com/training/index.html . Acesso em: 24 mai. 2023.
	W3SCHOOLS. W3Schools Online Web Tutorials, 2023. Java Tutorial. Disponível em: https://www.w3schools.com/java/default.asp . Acesso em: 24 mai. 2023.

Bibliografia - opcional

DEITEL, P.; DEITEL, H.; WALD, A. Android 6 para programadores uma abordagem baseada em aplicativos. São Paulo: Bookman 2016. (Livro Eletrônico)

LEAL, N. G. V. Dominando o Android: do básico ao avançado. 2. ed,. São Paulo: Novatec, 2015. 789 p. ISBN 9788575224120.

LECHETA, R. R. Google Android: aprenda a criar aplicações para dispositivos móveis com o Android SDK. 5. ed. São Paulo: Novatec, 2015



Bibliografia - opcional

DEITEL, H.; DEITEL, P. Java: como programar. 10. ed. São Paulo: Pearson Education do Brasil, 2017 (e-book).

DEITEL, Harvey M. Android : como programar. 2. Porto Alegre Bookman 2015. ISBN 9788582603482 (livro eletrônico).

GOOGLE Android Developers. Disponível em <https://developer.android.com/training/index.html>. Acesso em 06/08/2020.

GOOGLE. Kotlin: Primeiros Passos. Disponível em: <https://developer.android.com/kotlin/first>. Acesso em 06/08/2020.

LEE, V. Aplicações móveis: arquitetura, projeto e desenvolvimento. São Paulo: Pearson Makron Books, 2005. ISBN 9788534615402. (Livro Eletrônico)



Materiais extras no canal



<https://www.youtube.com/sosneuronios>



Materiais e exemplos

HTML, CSS, JavaScript

Softwares que utilizaremos

Nesta disciplina utilizaremos os seguintes softwares:

- Java SE SDK
- Android Studio + Android SDK + AVD (Android Virtual Device Manager ou Device Manager)

ou

- Android Studio + Android SDK + Visual Studio Emulator for Android

ou

- Android Studio + Android SDK + BlueStacks

Também:

- SQLite
- Multiplataforma: Cordova, Flutter

<https://visualstudio.microsoft.com/pt-br/vs/msft-android-emulator/>

www.bluestacks.com



Programação Orientada a Objetos (POO)

(breve revisão; pode desconsiderar os próximos slides)

Classes e objetos

- Uma **classe** é uma forma, molde, gabarito, a partir da qual podemos criar **objetos**;
- Uma **classe** descreve as características (estado ou informações + comportamento ou processos) de um determinado tipo ou **categoria de objetos**;
- Um **objeto** é uma **instância** ou **ocorrência** de determinada **classe**: um objeto será criado ou instanciado utilizando alguma declaração de classe existente;
- Utilizando uma **classe** podemos criar **N objetos**.
- Uma classe é composta por: **atributos** e **métodos**.

Métodos construtores de uma classe

// Exemplo na linguagem **Java**

```
class Pessoa { //declaração de uma classe
```

```
    private float salario;  
    private String nome;
```

```
    public Pessoa() {  
        salario = 0.0f;  
        nome = "sem nome";  
    }
```

```
    public Pessoa(String inN, float inS) {  
        salario = inS;  
        nome = inN;  
    }
```

```
} //fim da classe
```

//vamos criar dois objetos (instâncias da classe Pessoa):

```
Pessoa pa = new Pessoa(); //construtor sem parâmetros
```

```
Pessoa pb = new Pessoa("João Lopes", 2000.0f); //construtor com parâmetros
```

Encapsulamento (encapsulação)

- As características (dados, métodos) de uma classe são representadas ou descritas em forma agrupada, monolítica, como peças **encapsuladas** dentro da classe.
- Uma classe poderá estabelecer **categorias de visibilidade** ou **modificadores de acesso**, para especificar quais características (atributos, métodos) serão *privadas*, *públicas*, *protegidas* ou com *acesso de pacote*. Esta é uma possibilidade que permite *ocultar* determinadas características e *mostrar* outras.
- É considerada uma boa prática de programação O.O. declarar os atributos de uma classe com categoria **private** (privados) e fornecer, quando seja necessário, métodos **public** (públicos) para acessar estes dados.
 - consistência dos atributos + validar a informação de entrada + manter os atributos com valores e formatos válidos + mecanismos para conversão de formatos.
 - Uma convenção muito utilizada pelos programadores OO é utilizar, para cada dado membro (atributo da classe), um método **getAtributo** que permita acessar o valor do atributo e outro método com o nome **setAtributo** para alterar o valor do atributo.


```
public class Trabalhador {
```

```
private String nome;  
private float salario;  
private char sexo;
```

dados
privados

```
public Trabalhador() {  
    nome = "";  
    sexo = '-';  
    salario = 0;  
}
```

```
public Trabalhador (String inNome, char  
                    inSexo, float inSalario) {  
    setNome(inNome);  
    setSexo(inSexo);  
    setSalario(inSalario);  
}
```

```
public String getNome() {  
    return nome;  
}
```

```
public void setNome(String inNome) {  
    nome = inNome;  
}
```

```
public float getSalario() {  
    return salario;  
}
```

métodos get
e set
públicos

```
public String getSalarioStr(){  
    return ("R$" + salario);  
}
```

```
public void setSalario(float inSalario) {  
    if(inSalario>0)salario = inSalario;  
    else salario=0;  
}
```

```
public char getSexo() {  
    return sexo;  
}
```

validação

```
public void setSexo(char inSexo) {  
    sexo = '-';  
    if(inSexo=='F' || inSexo=='M') sexo = inSexo;  
}
```

Herança na P.O.O.

A herança é um relacionamento entre duas classes;

Uma destas classes será chamada de **classe base** (superclasse, classe pai, classe mãe) e a outra será chamada de **classe derivada** (subclasse, classe filha, classe herdeira);

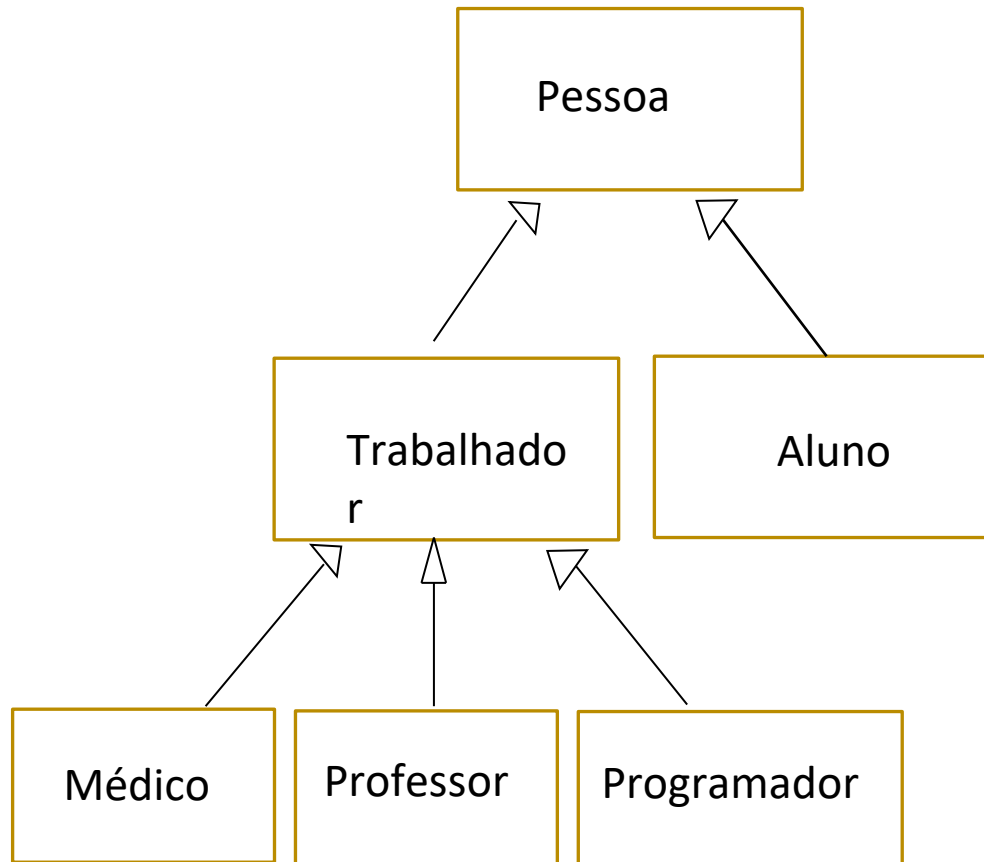
O relacionamento de herança permite representar **generalização / especificação** (particularização): uma classe base mais geral e uma classe derivada que será mais específica ou particular;

Caso especial: Herança de interface ou por implementação é outro caso de relacionamento de herança;

Na metodologia UML poderá ser utilizado um "diagrama de classes" para representar relacionamentos de herança entre classes (e outros tipos de relacionamentos que não sejam de herança, também);

Num diagrama de herança, as classes mais gerais ou abstratas aparecem perto da raiz (parte superior) e as classes mais particulares ou específicas aparecerão nas folhas (inferior).

Exemplo de diagrama de classes UML (herança)



Obs: Pessoa, Aluno, Trabalhador, Médico, Professor e Programador são classes.

Três características fundamentais da herança

- Uma classe derivada **herdará as características da classe base** (atributos + métodos);
- A classe derivada **poderá acrescentar (declarar) novos dados e métodos** que não existiam na classe base;
- A classe derivada **poderá redefinir métodos** que foram declarados na classe base.

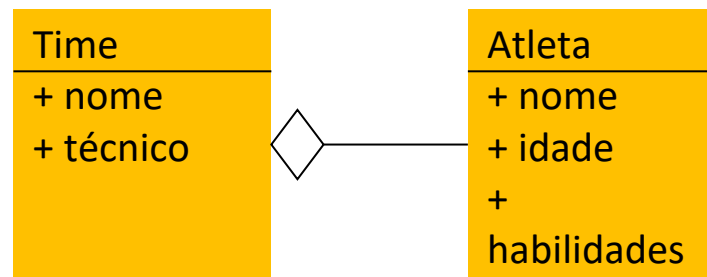
Importância da herança

- **Aproveitar classes** que já foram desenvolvidas, herdando seus atributos e métodos $\Rightarrow$ eficiência no desenvolvimento. Utilização de **pacotes de classes** fornecidos por **empresas de primeira linha** (por exemplo: Microsoft, Oracle, Google etc.).
- A **depuração de erros e manutenção** é mais simples utilizando classes e herança.
- A herança facilita a **expansão** fácil de um sistema.

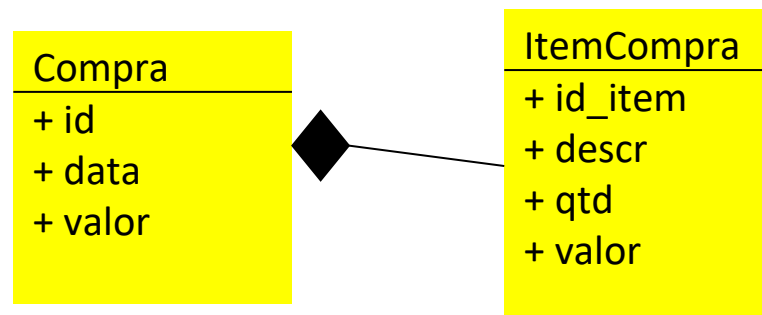
Agregação e Composição

São conceitos de associação entre classes, demonstram a relação do todo ser formado por partes, essa relação pode ser forte ou fraca.

Agregação, a existência do objeto parte faz sentido, mesmo não existindo o objeto todo.



Composição é uma agregação mais forte, ou seja, a existência do 'objeto da parte' **não** faz sentido se o 'objeto todo' não existir.



Agregação e Composição

Esses conceitos de associação podem ser implementados em Java como:

```
public class Pessoa {  
    private Endereco end;  
    ...  
    void Pessoa(Endereco endereco)  
    {  
        end = endereco;  
        ...  
    }  
}
```

Referência



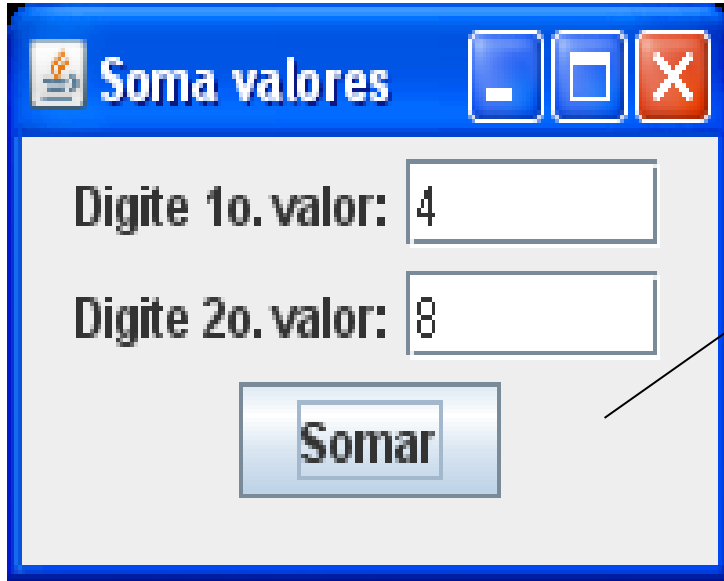
```
public class Endereco {  
    String rua;  
    String cep;  
    ...  
}
```

Classes aninhadas



```
public class Empresa {  
    Departamento depto [ ];  
    ...  
    ...  
    public class Departamento {...}  
        ...  
    }  
}
```

Uma janela com interface gráfica em Java SE => POO

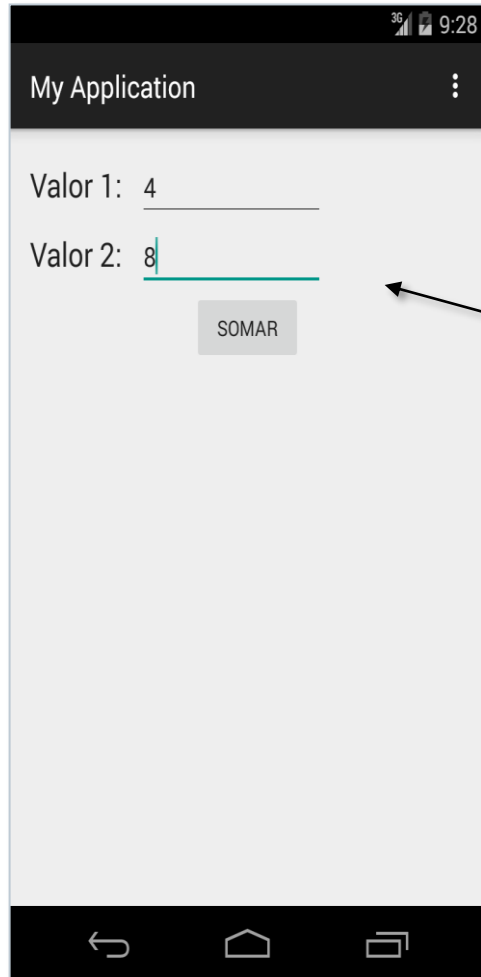


objetos que fazem parte do Visual desta janela => **composição**

- os objetos são criados utilizando, por exemplo, as classes JLabel, JTextField, JButton... do pacote swing
- a janela é uma classe derivada (**herança**) da classe JFrame

Isto seria em Java SE.
E como ficaria em Android?

Exemplo: a tela de um aplicativo Android => POO



- os objetos são criados utilizando as classes `TextView`, `EditText`, `Button` etc., que são classes do pacote `android.widget`
- a janela é uma classe derivada da classe `Activity` ou de outra classe semelhante